

The Master Guide: AI-Assisted Power Platform Engineering

This guide explains the complete architecture, tooling, and workflows of your local Pro-Dev Power Platform environment. It is designed to bridge the gap between Microsoft's cloud-first ecosystem and modern, local, AI-driven software engineering.

1. The Environment Overview

Historically, Power Platform development happened entirely in the web browser ("Citizen Development"). This environment changes that paradigm. By utilizing **VS Code**, **the Power Platform CLI (PAC)**, and **Git**, you have transformed your setup into an Enterprise "Pro-Dev" Workstation.

This allows you to treat Flows and Apps like real code: you can back them up, track changes, inject complex logic, and use AI Agents to automate the heavy lifting.

2. The Core Toolchain

- **Power Platform CLI (`pac`)**: The bridge between your laptop and the Microsoft Cloud. It allows you to download (clone), pack, and deploy solutions without using the browser.
 - **Git & GitHub**: Your version control system. Every time a flow is finished, it is committed here to ensure you never lose a piece of logic.
 - **PowerShell (`pwsh`)**: Used for writing automation scripts, such as sending fake API payloads to test your flows.
 - **Playwright**: A browser automation tool used for end-to-end testing (when API testing isn't possible).
 - **The AI Agent (Me!)**: Your local coding partner capable of reading downloaded flow JSON, rewriting complex architecture, and running terminal commands autonomously.
-

3. The "Shell & Inject" Agentic Workflow

Because Microsoft enforces strict security (preventing external bots from generating OAuth connection tokens), the AI cannot build a flow connecting to Dataverse or Outlook entirely from scratch. We use the **Shell & Inject** method to collaborate flawlessly:

Step 1: The Blueprint (AI)

You describe the goal. The AI designs the architecture (Tables, Columns, Flow Triggers) and provides a markdown blueprint.

Step 2: The Shell (Human)

You go to `make.powerapps.com` and act as the "Security Authenticator."

- Create the blank Dataverse tables.
- Create a new Flow and drop the **empty nodes** (Triggers and Actions) onto the canvas.
- Do not write any formulas. Hit Save. This forces Microsoft to securely wire up the "ID Badges" (Connection References).

Step 3: The Takeover (AI)

You return to VS Code and say, "The shell is ready."

- The AI uses `pac solution clone` to download the cloud code.
- The AI modifies the raw `.json` files locally, injecting complex JSON schemas, math formulas, `if()` statements, and data mapping.
- The AI uses `pac solution pack` and `pac solution import` to deploy the finished code back to the cloud.

Step 4: Autonomous Testing (AI)

The AI writes a PowerShell script (using `Invoke-RestMethod`) to instantly fire data at your new system to prove it works perfectly.

4. Architectural Best Practices

Webhooks over Emails

Relying on Outlook triggers ("When an email arrives") is prone to failure due to licensing limits (requiring Exchange mailboxes), spam filters, and missing attachments. **The Pro-Dev Standard:** Use the **"When an HTTP request is received"** trigger. This gives your Flow a secure Webhook URL. Third-party systems (like QuickBooks, vendor portals, or AI OCR scanners) can POST JSON data directly to this URL, processing data in milliseconds.

Avoiding Playwright for Cloud Authoring

While Playwright is installed, it should only be used for *Testing* user interfaces. Using it to *build* Flows by having a bot click "Add Action" in the Power Automate portal is highly discouraged. Microsoft's UI is dynamic, and bot-driven authoring frequently triggers CAPTCHAs and MFA lockouts.

5. Troubleshooting & Quirks

- **"Invalid Connection" on Outlook:** This means your developer tenant account (e.g., `user@onmicrosoft.com`) does not have a Microsoft 365 Exchange license attached to it. Switch to the HTTP architecture, or use a personal Outlook.com account.
 - **HTTP 401 Unauthorized during testing:** Modern Power Automate environments lock HTTP triggers to "Specific users in my tenant" by default. To test via simple PowerShell scripts, you must change this setting on the trigger block in the UI to **"Anyone"** to generate a URL with a secure Signature (`&sig=`).
 - **macOS Auth Broker Bug:** If `pac auth create` crashes on a Mac, always append `--deviceCode` to force the browser-based device login method.
-

Document automatically generated by your AI Agent.